EXPERIMENT 5

Implementation of Various Scheduling Mechanisms Using an Open-Source Cloud Simulator

1 Update Ubuntu

sudo apt update

Why: Refreshes Ubuntu's package index so the system can identify available package versions.

sudo apt upgrade -y

Why: Installs available updates for already-installed software. The -y flag automatically confirms the operation.

2 Install Java, Maven and Git

sudo apt install openjdk-17-jdk maven git -y

Why: Installs the Java compiler/runtime, Maven build tool and Git version-control utility.

java -version

Why: Checks the installed Java runtime version.

javac -version

Why: Checks that the Java compiler is available.

mvn -version

Why: Confirms Maven installation and displays the Java version Maven uses.

git --version

Why: Confirms Git installation.

Expected check: Java and javac should report version 17.x. Maven should display a version and show that it is using Java 17. Resolve any Java-version mismatch before proceeding.

3. Create the CloudSim Plus Maven project

3.1 Create folders

mkdir -p ~/CloudSchedulingLab/src/main/java/lab

Why: Creates the project directory and standard Maven Java source directory. -p creates missing parent directories too.

cd ~/CloudSchedulingLab

Why: Changes the terminal's current directory to the project root.

3.2 Create pom.xml

nano pom.xml

Why: Opens the nano text editor to create the Maven configuration file.

Enter the following content exactly:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>lab</groupId>
    <artifactId>cloud-scheduling-lab</artifactId>
    <version>1.0-SNAPSHOT</version>
    <properties>
      <maven.compiler.release>17</maven.compiler.release>
      <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>
    <dependencies>
      <dependency>
        <groupId>org.cloudsimplus</groupId>
        <artifactId>cloudsimplus</artifactId>
        <version>8.5.5</version>
      </dependency>
    </dependencies>
</project>
```

Save in nano: Ctrl+O, press Enter, then Ctrl+X.

Why this file is required: It defines the Maven project, sets Java 17 as the target and declares CloudSim Plus as a dependency. Maven downloads the required library files automatically.

3.3 Resolve dependencies

mvn dependency:resolve

Why: Downloads the declared CloudSim Plus library and supporting dependencies into Maven's local repository.

mvn clean

Why: Removes old build output (the target directory), ensuring the next build starts cleanly.

4. Implement the experiment

nano src/main/java/lab/SchedulingExperiment.java

Why: Creates the Java source file containing the simulation setup, scheduler configurations and output code.

Copy the complete Java program below. Preserve the package declaration and all class/method names.

```
package lab;

import java.util.ArrayList;
import java.util.List;

import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.datacenters.Datacenter;
import org.cloudbus.cloudsim.datacenters.DatacenterSimple;
import org.cloudbus.cloudsim.brokers.DatacenterBroker;
import org.cloudbus.cloudsim.brokers.DatacenterBrokerSimple;
import org.cloudbus.cloudsim.cloudlets.Cloudlet;
import org.cloudbus.cloudsim.cloudlets.CloudletSimple;
import org.cloudbus.cloudsim.hosts.Host;
import org.cloudbus.cloudsim.hosts.HostSimple;
import org.cloudbus.cloudsim.resources.Pe;
import org.cloudbus.cloudsim.resources.PeSimple;
import org.cloudbus.cloudsim.schedulers.cloudlet.CloudletScheduler;
import org.cloudbus.cloudsim.schedulers.cloudlet.CloudletSchedulerSpaceShared;
import org.cloudbus.cloudsim.schedulers.cloudlet.CloudletSchedulerTimeShared;
import org.cloudbus.cloudsim.schedulers.vm.VmScheduler;
import org.cloudbus.cloudsim.schedulers.vm.VmSchedulerSpaceShared;
import org.cloudbus.cloudsim.schedulers.vm.VmSchedulerTimeShared;
import org.cloudbus.cloudsim.utilizationmodels.UtilizationModel;
import org.cloudbus.cloudsim.utilizationmodels.UtilizationModelFull;
```

```
import org.cloudbus.cloudsim.vms.Vm;
import org.cloudbus.cloudsim.vms.VmSimple;
import org.cloudbus.cloudsim.provisioners.ResourceProvisionerSimple;
import org.cloudbus.cloudsim.allocationpolicies.VmAllocationPolicySimple;

public class SchedulingExperiment {
    private static final int HOST_PES = 4;
    private static final int VM_COUNT = 2;
    private static final int CLOUDLET_COUNT = 4;
    private static final long MIPS = 1000;
    private static final long CLOUDLET_LENGTH = 20000;

    public static void main(String[] args) {
        runScenario("A: VM SpaceShared + Cloudlet SpaceShared",
                new VmSchedulerSpaceShared(),
                new CloudletSchedulerSpaceShared());
        runScenario("B: VM SpaceShared + Cloudlet TimeShared",
                new VmSchedulerSpaceShared(),
                new CloudletSchedulerTimeShared());
        runScenario("C: VM TimeShared + Cloudlet SpaceShared",
                new VmSchedulerTimeShared(),
                new CloudletSchedulerSpaceShared());
        runScenario("D: VM TimeShared + Cloudlet TimeShared",
                new VmSchedulerTimeShared(),
                new CloudletSchedulerTimeShared());
    }

    private static void runScenario(String title,
                                    VmScheduler vmScheduler,
                                    CloudletScheduler cloudletScheduler) {
        System.out.println("\n================================================");
        System.out.println(title);
        System.out.println("================================================");

        CloudSim simulation = new CloudSim();
        createDatacenter(simulation, vmScheduler);
        DatacenterBroker broker = new DatacenterBrokerSimple(simulation);

        List<Vm> vmList = createVms(cloudletScheduler);
        List<Cloudlet> cloudletList = createCloudlets();

        broker.submitVmList(vmList);
        broker.submitCloudletList(cloudletList);
        simulation.start();

        List<Cloudlet> finished = broker.getCloudletFinishedList();
        System.out.printf("%-10s %-10s %-12s %-12s %-12s%n",
                "Cloudlet", "VM", "Status", "Start", "Finish");
        for (Cloudlet cloudlet : finished) {
            System.out.printf("%-10d %-10d %-12s %-12.2f %-12.2f%n",
                    cloudlet.getId(), cloudlet.getVm().getId(),
                    cloudlet.getStatus(), cloudlet.getExecStartTime(),
                    cloudlet.getFinishTime());
        }
        System.out.println("Completed Cloudlets: " + finished.size()
                + " / " + CLOUDLET_COUNT);
    }

    private static Datacenter createDatacenter(CloudSim simulation,
                                               VmScheduler vmScheduler) {
        List<Pe> peList = new ArrayList<>();
        for (int i = 0; i < HOST_PES; i++) {
            peList.add(new PeSimple(MIPS));
        }
        Host host = new HostSimple(8192, 100000, 1000000, peList);
        host.setRamProvisioner(new ResourceProvisionerSimple());
```

```
        host.setBwProvisioner(new ResourceProvisionerSimple());
        host.setVmScheduler(vmScheduler);

        List<Host> hostList = new ArrayList<>();
        hostList.add(host);
        return new DatacenterSimple(simulation, hostList,
                new VmAllocationPolicySimple());
    }

    private static List<Vm> createVms(CloudletScheduler scheduler) {
        List<Vm> vmList = new ArrayList<>();
        for (int i = 0; i < VM_COUNT; i++) {
            Vm vm = new VmSimple(i, MIPS, 2);
            vm.setRam(2048).setBw(10000).setSize(10000);
            vm.setCloudletScheduler(scheduler);
            vmList.add(vm);
        }
        return vmList;
    }

    private static List<Cloudlet> createCloudlets() {
        List<Cloudlet> cloudletList = new ArrayList<>();
        UtilizationModel utilization = new UtilizationModelFull();
        for (int i = 0; i < CLOUDLET_COUNT; i++) {
            Cloudlet cloudlet = new CloudletSimple(
                    CLOUDLET_LENGTH, 1, utilization);
            cloudlet.setSizes(300);
            cloudletList.add(cloudlet);
        }
        return cloudletList;
    }
}
```

Save: Ctrl+O → Enter → Ctrl+X.

5. Compile and execute

5.1 Compile

mvn clean compile

Why: Compiles the Java source code and reports syntax or API errors before running the simulation.

5.2 Run

mvn exec:java -Dexec.mainClass="lab.SchedulingExperiment" -Dexec.classpathScope=compile

Why: Executes the main() method with Maven and the project dependency classpath.

If the exec plugin prefix is not resolved, run the fully qualified plugin command:

mvn org.codehaus.mojo:exec-maven-plugin:3.5.0:java -Dexec.mainClass="lab.SchedulingExperiment" -Dexec.classpathScope=compile

Why: Invokes a specific Maven Exec plugin version directly.